LLM-Assisted Compiler Error Diagnosis & Repair Tool

Step-by-Step Terminal Commands, Architecture Highlights & Faculty Talking Points

Project Review Guide
BCSE307L — Compiler Design

Student: Namish (3rd Year CSE)
Stack: Python, Automata, AST, Groq Cloud API

1. Executive Summary & Core Novelty

Core Research Hypothesis: Existing LLM-assisted repair systems (e.g., wrappers around GCC or Clang) treat compilers as black boxes by scraping flattened stderr text strings after syntactic and semantic structure has been lost. This project implements a custom C-subset compiler front-end where every error is modeled as an enriched, typed **Diagnostic** object directly at the moment of failure. These diagnostics carry live grammar rule stack paths, delimiter opener references, and scoped symbol table snapshots into **error-type-specific prompt templates**, which are dispatched to **Groq Cloud API** (e.g., openai/gpt-oss-120b, llama-3.1-8b) and compiled in memory through an **automated fix verification layer**.

2. Step-by-Step Review Demonstration Script

Step 1: Comprehensive Test Suite Verification (76/76 Passing)

```
$ python -m unittest discover -s tests -v
```

What this command executes: Discovers and runs all 76 unit tests across automata minimization, lexer error recovery, recursive-descent precedence, semantic type checking, Groq/LLM integration, fix verification, and evaluation.

Compiler internals demonstrated: Automated test harness, regression testing, full pipeline verification (100% test pass rate across 76 tests).

What to say to faculty: *"We verified our compiler front-end with 76 unit tests built from first principles. Every phase—from regex DFA construction to panic-mode recovery, semantic scope checking, and Groq LLM integration—is covered with zero failing tests."*

Step 2: Automata Theory: Thompson's NFA to Minimal DFA

```
$ .\toycompiler.bat dfa-stats
```

What this command executes: Compiles the formal lexical specification into an ε-NFA, determinizes it into a DFA, and applies Moore's partition refinement to minimize states, printing state, accepting, and transition metrics.

Compiler internals demonstrated: Regex AST parsing, Thompson's Construction, Subset Construction (ε-closure), Moore's Partition Minimization (67 states, 62 accepting, 2,730 transitions).

What to say to faculty: *"Instead of using standard high-level regex libraries, our lexer compiles token specifications using classic automata theory. It transforms regular expressions into an ε-NFA, converts them via subset construction, and minimizes the DFA to an optimal 67 states."*

Step 3: Recursive-Descent Parser, AST & Scoped Symbol Table

```
$ .\toycompiler.bat run examples\valid_factorial.toy --ast --symbols
```

What this command executes: Tokenizes and parses a complete Toy-C program with recursion, loops, and boolean logic; renders an ASCII Abstract Syntax Tree and prints the scoped symbol table tree.

Compiler internals demonstrated: Explicit operator precedence hierarchy (assignment → or → and → equality → comparison → term → factor → unary → call → primary); nested lexical scopes (global, function, block).

What to say to faculty: *"Our parser implements recursive-descent with explicit grammar layering to enforce operator precedence without ambiguity. The semantic analyzer walks the AST and builds a hierarchical scope tree tracking variable names, types, and function signatures."*

Step 4: Panic-Mode Multi-Error Recovery & Rule Stack Tracing

```
$ .\toycompiler.bat run examples\err_multiple.toy
```

What this command executes: Parses a file containing multiple syntax errors in different declarations; recovers at statement and declaration boundaries (';', '}', keywords) to diagnose all errors in a single pass.

Compiler internals demonstrated: Panic-mode recovery, synchronization tokens, live grammar rule stack tracking attached to Diagnostic objects.

What to say to faculty: *"Rather than aborting on the first syntax error, Toy-C implements panic-mode synchronization. It recovers at delimiter and statement boundaries so students receive a comprehensive diagnostic report in a single compilation run."*

Step 5: Error-Type-Specific Prompting & Verified Repair (Syntax Error)

```
$ .\toycompiler.bat diagnose examples\err_missing_semicolon.toy --online
```

What this command executes: Extracts the error context (source window, insertion caret, rule stack), dispatches the specialized 'PromptSyntaxMissingDelimiter' template, queries Groq Cloud API for structured JSON diagnosis, recompiles the suggested patch in memory, and prints a verified diff.

Compiler internals demonstrated: Structured Diagnostic → Context Extraction → Typed Prompt Template → Live Groq Inference (openai/gpt-oss-120b) → In-Memory Recompilation → [VERIFIED FIX] badge.

What to say to faculty: *"Here is our core novelty: the diagnostic knows the exact insertion point where ';' was expected. It dispatches the specialized PromptSyntaxMissingDelimiter template to Groq's high-speed cloud inference. Once the LLM suggests a fix, our verification layer recompiles the code in memory to guarantee the fix works before showing it to the student."*

Step 6: Semantic Typo Diagnosis with In-Scope Symbol Snapshot

```
$ .\toycompiler.bat diagnose examples\benchmarks\12_sem_var_typo.toy --online
```

What this command executes: Diagnoses an undeclared variable caused by a typo ('conter' instead of 'counter'); dispatches 'PromptUndeclaredVariable' with active in-scope symbols; queries Groq and verifies the clean recompilation.

Compiler internals demonstrated: Scope traversal, difflib spelling recommendations, in-scope symbol table slice injection into LLM prompt context.

What to say to faculty: *"For semantic errors like undeclared variables, our compiler captures a snapshot of all active symbols in scope. The prompt template passes this symbol table slice to Groq along with spelling distance metrics, allowing the LLM to fix typos with 100% precision."*

Step 7: Machine-Readable Structured JSON Diagnostic Output

```
$ .\toycompiler.bat run examples\benchmarks\12_sem_var_typo.toy --json
```

What this command executes: Emits the complete compiler diagnostic as machine-readable JSON, showing error category, code, rule stack, source snippet, expected/found tokens, details, and visible symbols.

Compiler internals demonstrated: Rich Diagnostic schema: code, category, location, rule_stack, details, symbols, hint, related locations.

What to say to faculty: *"Notice that our error representation is structured data from the exact moment of failure. There is zero fragile text parsing or stderr scraping; downstream agents and prompt templates receive full compiler context."*

Step 8: Comparative Evaluation Suite & Ablation Study

```
$ .\toycompiler.bat eval --output evaluation_report.md --json
```

What this command executes: Executes a 20-program benchmark suite; runs Evaluation 1 (Toy-C structured context vs. raw GCC/Clang stderr) and Evaluation 2 (typed prompt dispatching vs. generic single-prompt baseline); generates a Markdown report.

Compiler internals demonstrated: Empirical evaluation, ablation study, rubric scoring (novice readability, grounding, actionability), fix verification rate.

What to say to faculty: *"To validate our contribution scientifically, we curated 20 benchmark programs across syntax, lexical, and semantic error classes. Our ablation study proves that error-type-specific prompting combined with in-memory recompilation achieves a 100% clean fix rate."*

3. Quick-Reference Review Cheat Sheet

Step	Command	Key Compiler Concept Demonstrated	Target Faculty Question Answered
1	`python -m unittest discover -s tests -v`	76/76 passing tests across all phases	"Is the implementation thoroughly tested?"
2	`.\toycompiler.bat dfa-stats`	Moore's minimization (67 states, 62 accepting)	"How did you implement lexical analysis?"
3	`.\toycompiler.bat run examples\valid_factorial.toy --ast --symbols`	Precedence AST + Scoped Symbol Table	"How is grammar precedence and scope handled?"
4	`.\toycompiler.bat run examples\err_multiple.toy`	Panic-mode statement synchronization	"Does the parser recover from syntax errors?"
5	`.\toycompiler.bat diagnose examples\err_missing_semicolon.toy --online`	Groq LLM delimiter repair + [VERIFIED FIX] diff	"How does LLM repair work with verification?"
6	`.\toycompiler.bat diagnose examples\benchmarks\12_sem_var_typo.toy --online`	Symbol table slice injection + typo repair	"What context does the LLM receive?"
7	`.\toycompiler.bat run examples\benchmarks\12_sem_var_typo.toy --json`	Typed Diagnostic JSON schema	"How is internal compiler state exported?"
8	`.\toycompiler.bat eval --output evaluation_report.md --json`	20-program comparative study & ablation	"What is the quantitative evaluation result?"